



البرمجة بلغة ++C

جنى الخطاب

Control Structures I (Selection)

Control Structures

- A computer can proceed:

1. In sequence

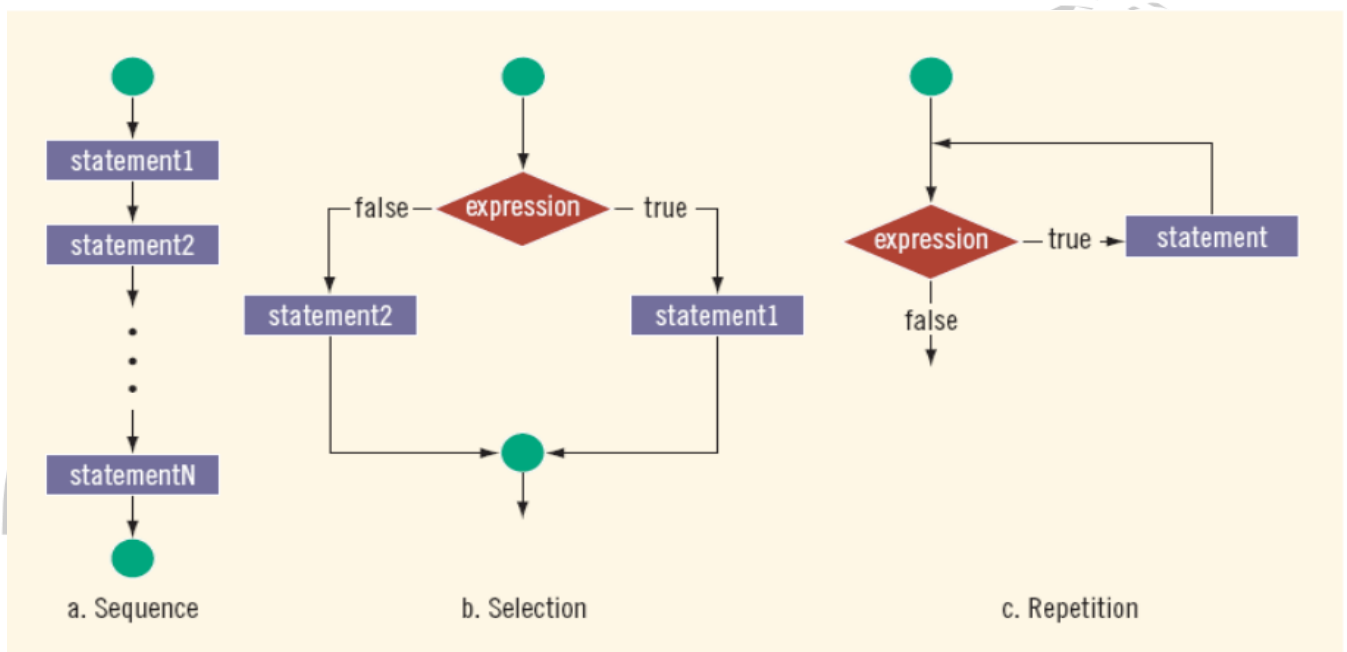
الكمبيوتر ممكن ينفذ بثلاث طرق الأولى التتابع (جمل متتابعة).

2. Selectively (branch) - making a choice

بشكل انتقائي يعني ممكن ينفذ الجملة ممكن لأ) حسب الشرط.

– Repetitively (iteratively) - looping

بشكل متكرر يعني أن يكرر تنفيذ جملة أو مجموعة من الجمل) يكرر التنفيذ طالما الشرط يتحقق.



- Some statements are executed only if certain conditions are met

بعض الجمل لا تنفذ الا اذا تحققت شروط معينة كما في selection and repetition.

- A condition is met if it evaluates to true

الشرط يتحقق اذا قيمه الكومبيلر على انه (true . ال true & false نوعهم). (boolean)

Relational Operators

A condition is represented by a logical (Boolean) expression that can be **true** or **false** .

الشرط يعبر عنه بتعبير منطقي (نوع بياناته **boolean**) يمكن أن يكون **true** أو **false**

- Relational operators:

TABLE 4-1 Relational Operators in C++

Operator	Description
==	equal to
!=	not equal to
<	less than
<=	less than or equal to
>	greater than
>=	greater than or equal to

ال relational operators الموضحين في الجدول:

- Allow comparisons

للمقارنة

- Require two operands (binary)

يتطلب عاملين (أكيد عشان نقارنهم في بعض)

- Evaluate to true or false

يقيم ال expression المكتوب فيهم اما **true** أو **false**

- You can use the relational operators with all three simple data types:

نقدر نستخدمها مع كل أنواع البيانات ال (simple اللي هم integral & floating-point ، اخر نوع Enumeration type ما رح نستخدمه في كل المادة).

أمثلة:

- $8 < 15$ evaluates to **true**

- $6 != 6$ evaluates to **false**

- $2.5 > 5.8$ evaluates to **false**

- $5.9 <= 7.5$ evaluates to **true**

Comparing Floating-Point Numbers for Equality

- Comparison of floating-point numbers for equality may not behave as you would expect

مقارنة نوع floating-point ممكن ما يعطي النتيجة اللي بتوقعها نتيجة قلة الدقة في حسابات هذا النوع) هاي نقطة مش كثير مهمة).

Comparing Characters

كيف نقارن نوع الـ characters؟

في شيء اسمه ASCII value لكل character، تكون المقارنة بين الـ ASCII values.

ASCII value of 'a' = 97, 'b' = 98, 'c' = 99, 'z' = 122.

ASCII value of 'A': 65, 'B': 66, 'C': 67, 'Z': 90

ASCII value of '0': 48, '1': 49, '2': 50, '9': 57

غالبا تعطى في السؤال.

أمثلة:

Expression	Value of Expression	Explanation
' ' < 'a'	true	The ASCII value of ' ' is 32, and the ASCII value of 'a' is 97. Because 32 < 97 is true, it follows that ' ' < 'a' is true.
'R' > 'T'	false	The ASCII value of 'R' is 82, and the ASCII value of 'T' is 84. Because 82 > 84 is false, it follows that 'R' > 'T' is false.
'+' < '*'	false	The ASCII value of '+' is 43, and the ASCII value of '*' is 42. Because 43 < 42 is false, it follows that '+' < '*' is false.
'6' <= '>'	true	The ASCII value of '6' is 54, and the ASCII value of '>' is 62. Because 54 <= 62 is true, it follows that '6' <= '>' is true.

Relational Operators and the string Type

Strings are compared character by character, starting with the first character, comparison continues until either a mismatch is found or all characters are found equal.

كيف نقارن نوع الـ string؟

نبش من أول character لكل string ونقارنهم حسب الـ ASCII value اذا اختلفوا القيم بنقرر اما اذا تساوا بنقارن الـ character الثاني بالتاني وهكذا لحد ما نصل الى اختلاف.

- Suppose we have the following declarations:

```
string str1 = "Hello";
```

```
string str2 = "Hi";
```

```
string str3 = "Air";
```

```
string str4 = "Bill";
```

```
string str5 = "Big";
```

Expression	Value	Explanation
<code>str1 < str2</code>	<code>true</code>	<code>str1 = "Hello"</code> and <code>str2 = "Hi"</code> . The first characters of <code>str1</code> and <code>str2</code> are the same, but the second character 'e' of <code>str1</code> is less than the second character 'i' of <code>str2</code> . Therefore, <code>str1 < str2</code> is <code>true</code> .
<code>str1 > "Hen"</code>	<code>false</code>	<code>str1 = "Hello"</code> . The first two characters of <code>str1</code> and <code>"Hen"</code> are the same, but the third character 'l' of <code>str1</code> is less than the third character 'n' of <code>"Hen"</code> . Therefore, <code>str1 > "Hen"</code> is <code>false</code> .
<code>str3 < "An"</code>	<code>true</code>	<code>str3 = "Air"</code> . The first characters of <code>str3</code> and <code>"An"</code> are the same, but the second character 'i' of <code>"Air"</code> is less than the second character 'n' of <code>"An"</code> . Therefore, <code>str3 < "An"</code> is <code>true</code> .
<code>str1 == "hello"</code>	<code>false</code>	<code>str1 = "Hello"</code> . The first character 'H' of <code>str1</code> is less than the first character 'h' of <code>"hello"</code> because the ASCII value of 'H' is 72, and the ASCII value of 'h' is 104. Therefore, <code>str1 == "hello"</code> is <code>false</code> .
<code>str3 <= str4</code>	<code>true</code>	<code>str3 = "Air"</code> and <code>str4 = "Bill"</code> . The first character 'A' of <code>str3</code> is less than the first character 'B' of <code>str4</code> . Therefore, <code>str3 <= str4</code> is <code>true</code> .
<code>str2 > str4</code>	<code>true</code>	<code>str2 = "Hi"</code> and <code>str4 = "Bill"</code> . The first character 'H' of <code>str2</code> is greater than the first character 'B' of <code>str4</code> . Therefore, <code>str2 > str4</code> is <code>true</code> .

<code>str4 >= "Billy"</code>	<code>false</code>	<code>str4 = "Bill"</code> . It has four characters and <code>"Billy"</code> has five characters. Therefore, <code>str4</code> is the shorter string. All four characters of <code>str4</code> are the same as the corresponding first four characters of <code>"Billy"</code> , and <code>"Billy"</code> is the larger string. Therefore, <code>str4 >= "Billy"</code> is <code>false</code> .
<code>str5 <= "Bigger"</code>	<code>true</code>	<code>str5 = "Big"</code> . It has three characters and <code>"Bigger"</code> has six characters. Therefore, <code>str5</code> is the shorter string. All three characters of <code>str5</code> are the same as the corresponding first three characters of <code>"Bigger"</code> , and <code>"Bigger"</code> is the larger string. Therefore, <code>str5 <= "Bigger"</code> is <code>true</code> .

في آخر فرعين في كلمة اطول من الثانية والمقارنة لحد آخر character في القصيرة تساوت القيم في هذه الحالة تكون الكلمة الاطول هي الاكبر.

Logical (Boolean) Operators and Logical Expressions

Operator	Description
<code>!</code> ← unary	not
<code>&&</code> ← binary	and
<code> </code> ← binary	or

Logical (Boolean) operators enable you to combine logical expressions (true or false). logical operators (الموضحين في الجدول يجمعون) logical expression (المقيمين الى). (true or false)

1.The not (!) operator

Expression	!(Expression)
<code>true</code> (nonzero)	<code>false</code> (0)
<code>false</code> (0)	<code>true</code> (1)

!بتعكس يعني اذا كان `true` بصير `false` وهكذا) أي رقم `non-zero` يقيّم كـ `true` يعني 1, -1, 34, انتبهوا انه `non-zero` يعني السالب برضه يقيم كـ `true` ، فقط العدد صفر يقيم كـ `false`). أمثلة:

-4 , (4-) إتقيّم `true` لأنها `non-zero` بالتالي `!(true)` تقيّم `false`.

EXAMPLE 4-2

Expression	Value	Explanation
<code>! ('A' > 'B')</code>	<code>true</code>	Because <code>'A' > 'B'</code> is <code>false</code> , <code>! ('A' > 'B')</code> is <code>true</code> .
<code>! (6 <= 7)</code>	<code>false</code>	Because <code>6 <= 7</code> is <code>true</code> , <code>! (6 <= 7)</code> is <code>false</code> .

2.The and (&&) operator

Expression1	Expression2	Expression1 && Expression2
<code>true (nonzero)</code>	<code>true (nonzero)</code>	<code>true (1)</code>
<code>true (nonzero)</code>	<code>false (0)</code>	<code>false (0)</code>
<code>false (0)</code>	<code>true (nonzero)</code>	<code>false (0)</code>
<code>false (0)</code>	<code>false (0)</code>	<code>false (0)</code>

معناها اذا ال expression الاول **او** ال expression الثاني يقيّموا انهم true يكون ال expression1 && expression2 يقيّم انه true ، اذا عالأقل واحد منهم كان false يكون false.

EXAMPLE 4-3

Expression	Value	Explanation
<code>(14 >= 5) && ('A' < 'B')</code>	<code>true</code>	Because <code>(14 >= 5)</code> is <code>true</code> , <code>('A' < 'B')</code> is <code>true</code> , and <code>true && true</code> is <code>true</code> , the expression evaluates to <code>true</code> .
<code>(24 >= 35) && ('A' < 'B')</code>	<code>false</code>	Because <code>(24 >= 35)</code> is <code>false</code> , <code>('A' < 'B')</code> is <code>true</code> , and <code>false && true</code> is <code>false</code> , the expression evaluates to <code>false</code> .

3.The or

(||) operator

Expression1	Expression2	Expression1 Expression2
<code>true (nonzero)</code>	<code>true (nonzero)</code>	<code>true (1)</code>
<code>true (nonzero)</code>	<code>false (0)</code>	<code>true (1)</code>
<code>false (0)</code>	<code>true (nonzero)</code>	<code>true (1)</code>
<code>false (0)</code>	<code>false (0)</code>	<code>false (0)</code>

معناها اذا ال expression الاول **او** ال expression الثاني يقيّم انه true يكون ال expression1 || expression2 يقيّم انه true ، اذا عالأقل واحد منهم كان true يكون true.

EXAMPLE 4-4

Expression	Value	Explanation
<code>(14 >= 5) ('A' > 'B')</code>	<code>true</code>	Because <code>(14 >= 5)</code> is <code>true</code> , <code>('A' > 'B')</code> is <code>false</code> , and <code>true false</code> is <code>true</code> , the expression evaluates to <code>true</code> .
<code>(24 >= 35) ('A' > 'B')</code>	<code>false</code>	Because <code>(24 >= 35)</code> is <code>false</code> , <code>('A' > 'B')</code> is <code>false</code> , and <code>false false</code> is <code>false</code> , the expression evaluates to <code>false</code> .
<code>('A' <= 'a') (7 != 7)</code>	<code>true</code>	Because <code>('A' <= 'a')</code> is <code>true</code> , <code>(7 != 7)</code> is <code>false</code> , and <code>true false</code> is <code>true</code> , the expression evaluates to <code>true</code> .

Order of Precedence



Operators	Precedence
<code>!, +, -</code> (unary operators)	first
<code>*, /, %</code>	second
<code>+, -</code>	third
<code><, <=, >=, ></code>	fourth
<code>==, !=</code>	fifth
<code>&&</code>	sixth
<code> </code>	seventh
<code>=</code> (assignment operator)	last

طبعا في حال تواجد الأقواس تكون الأولوية لها أولا.

EXAMPLE 4-5

Suppose you have the following declarations:

```
bool found = true;
bool flag = false;
int num = 1;
double x = 5.2;
double y = 3.4;
int a = 5, b = 8;
int n = 20;
char ch = 'B';
```

Expression	Value	Explanation
!found	false	Because found is true, !found is false.
x > 4.0	true	Because x is 5.2 and 5.2 > 4.0 is true, the expression x > 4.0 evaluates to true.
!num	false	Because num is 1, which is nonzero, num is true and so !num is false.
!found && (x >= 0)	false	In this expression, !found is false. Also, because x is 5.2 and 5.2 >= 0 is true, x >= 0 is true. Therefore, the value of the expression !found && (x >= 0) is false && true, which evaluates to false.
!(found && (x >= 0))	false	In this expression, found && (x >= 0) is true && true, which evaluates to true. Therefore, the value of the expression !(found && (x >= 0)) is !true, which evaluates to false.
x + y <= 20.5	true	Because x + y = 5.2 + 3.4 = 8.6 and 8.6 <= 20.5, it follows that x + y <= 20.5 evaluates to true.

إذا لاحظتوا في $(20.5 \geq x+y)$ ما في داعي للأقواس بين $x+y$ لأن الأولوية للجمع قبل إشارة الأصغر حسب الجدول فوق.

Expression	Value	Explanation
(n >= 0) && (n <= 100)	true	Here n is 20. Because 20 >= 0 is true, n >= 0 is true. Also, because 20 <= 100 is true, n <= 100 is true. Therefore, the value of the expression (n >= 0) && (n <= 100) is true && true, which evaluates to true.
('A' <= ch && ch <= 'Z')	true	In this expression, the value of ch is 'B'. Because 'A' <= 'B' is true, 'A' <= ch evaluates to true. Also, because 'B' <= 'Z' is true, ch <= 'Z' evaluates to true. Therefore, the value of the expression ('A' <= ch && ch <= 'Z') is true && true, which evaluates to true.

```
(a + 2 <= b) && !flag true
```

Now $a + 2 = 5 + 2 = 7$ and b is 8.
Because $7 <= 8$ is **true**, the expression
 $a + 2 <= b$ evaluates to **true**. Also, because
 $flag$ is **false**, $!flag$ is **true**. Therefore,
the value of the expression $(a + 2 <= b)$
&& $!flag$ is **true && true**, which
evaluates to **true**.

Short-Circuit Evaluation: evaluation of a logical expression stops as soon as the value of the expression is known.

تقييم تعبير منطقي يتوقف بمجرد معرفة قيمة التعبير.

كيف؟ يعني مثلاً عندي two expressions وبينهم $or(||)$ واول expression كان true فعلياً ما في داعي اشيك على الثاني لانه كان true أو false النتيجة وحدة اللي هي true. يعني الكومبايلر في هاي الحالة ما بيقرا ال expression الثاني أصلاً لغاية السرعة.
مثال:

```
2+4<10 || 5<=0
```

int Data Type and Logical (Boolean) Expressions

Earlier versions of C++ did not provide built-in data types that had Boolean values

- Logical expressions evaluate to either 1 or 0
- The value of a logical expression was stored in a variable of the data type int
- The identifier true has the value 1
- The identifier false has the value 0

الاصدارات القديمة من البرمجة ما كان فيها نوع بيانات boolean ، كان يعبر عنها ب int وكان ال logical expressions يقيموا اما الى واحد أو صفر بدل true و false. القيم true و false الهم قيم int ، لل true واحد ولل false صفر.

Logical (Boolean) Expressions

عادة في الرياضيات نكتب المتباينة المركبة (الثلاثية) بهذا الشكل:

```
0 <= num <= 10
```

لكن كيف الكومبايلر بيعمل الها evaluation من اليسار الى اليمين ابتداء ب $0 <= num$ بغض النظر عن قيمة المتغير num هذا ال expression قيمته اما واحد أو صفر اللي هو اما true أو false في الحالتين الصفر والواحد أقل من العشرة فال expression كامل رح يكون دائماً true. بالتالي ما اعطى النتيجة اللي بدنا ياها، الطريقة الصحيحة لكتابته هي كالتالي:

```
0 <= num && num <= 10
```

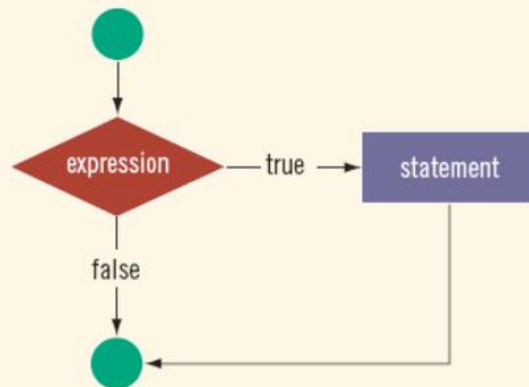
One-Way Selection

The syntax of one-way selection is:

```
if (expression)
    statement
```

الجملة تنفذ فقط اذا ال expression يقيم ك true اذا false تنتقل للجملة اللي بعدها.

if is a reserved word.



ex:

```
if(score >= 60)
    grade = 'P';
```

EXAMPLE 4-10

The following C++ program finds the absolute value of an integer:

```
//Program: Absolute value of an integer

#include <iostream>

using namespace std;

int main()
{
    int number, temp;

    cout << "Line 1: Enter an integer: "; //Line 1
    cin >> number; //Line 2
    cout << endl; //Line 3

    temp = number; //Line 4

    if (number < 0) //Line 5
        number = -number; //Line 6

    cout << "Line 7: The absolute value of "
         << temp << " is " << number << endl; //Line 7

    return 0;
}
```

Sample Run: In this sample run, the user input is shaded.

```
Line 1: Enter an integer: -6734
Line 7: The absolute value of -6734 is 6734
```

EXAMPLE 4-11

Consider the following statement:

```
if score >= 60      //syntax error
    grade = 'P';
```

This statement illustrates an incorrect version of an **if** statement. The parentheses around the logical expression are missing, which is a syntax error.

EXAMPLE 4-12

Consider the following C++ statements:

```
if (score >= 60);      //Line 1
    grade = 'P';      //Line 2
```

Because there is a semicolon at the end of the expression (see Line 1), the **if** statement in Line 1 terminates. The action of this **if** statement is null, and the statement in Line 2 is not part of the **if** statement in Line 1. Hence, the statement in Line 2 executes regardless of how the **if** statement evaluates.

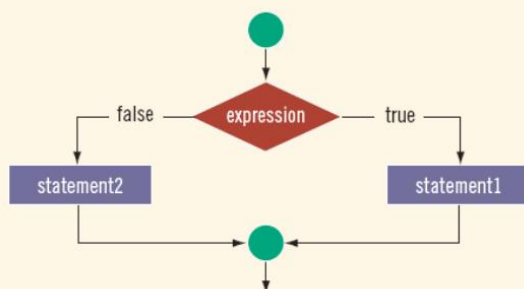
Two-Way Selection

- The syntax of two-way selection is:

```
if (expression)
    statement1
else
    statement2
```

إذا ال expression قيمته true بنفذ statement 1 إذا false بنفذ statement 2 عشان هيك اسمه two-way لأنه في الحالتين في تنفيذ.

else is a reserved word.



EXAMPLE 4-13

Consider the following statements:

```
if (hours > 40.0)           //Line 1
    wages = 40.0 * rate +
        1.5 * rate * (hours - 40.0); //Line 2
else                         //Line 3
    wages = hours * rate;    //Line 4
```

If the value of the variable `hours` is greater than 40.0, then the `wages` include overtime payment. Suppose that `hours` is 50. The expression in the `if` statement, in Line 1, evaluates to `true`, so the statement in Line 2 executes. On the other hand, if `hours` is 30, or any number less than or equal to 40, the expression in the `if` statement, in Line 1, evaluates to `false`. In this case, the program skips the statement in Line 2 and executes the statement in Line 4—that is, the statement following the reserved word `else` executes.

EXAMPLE 4-14

The following statements show an example of a syntax error:

```
if (hours > 40.0);           //Line 1
    wages = 40.0 * rate +
        1.5 * rate * (hours - 40.0); //Line 2
else                         //Line 3
    wages = hours * rate;    //Line 4
```

The semicolon at the end of the `if` statement (see Line 1) ends the `if` statement, so the statement in Line 2 separates the `else` clause from the `if` statement. That is, `else` is all by itself. Because there is no stand-alone `else` statement in C++, this code generates a syntax error.

انتبهوا دائما لل `syntax` اذا صحيح أو لا لأن في أسئلة مثل هذا السؤال اذا ما انتهت انه ال `syntax` غلط وجاوبت على هذا الأساس رح تلاقي جوابك ضمن الخيارات.

Compound (Block of) Statement

```
{
    statement1
    statement2
    .
    .
    .
    statementn
}
```

اذا مثلا بدنا يتنقذ أكثر من جملة مثلا ل `if statement` بمعينة بإمكاننا استخدام `braces { }`.

- A compound statement is a single statement

مثال:

```
if (age > 18)
{
    cout << "Eligible to vote." << endl;
    cout << "No longer a minor." << endl;
}
else
{
    cout << "Not eligible to vote." << endl;
    cout << "Still a minor." << endl;
}
```

Multiple Selections: Nested if

- Nesting: one control statement in another.

مثال:

```
if ( x > 0)
if(x % 2 == 0)
x = 4;
```

هذا الكود اذا قيمة x اكبر من صفر ينتقل الى if الثانية ويتحقق من الشرط اذا باقى القسمة على 2 يساوي صفر رح يعدل قيمة x وتصبح 4.

فعليا هون في statement وحدة لأن كل statement يجب أن تنتهي ب. semicolon. فاذا الشرط الاول false ما رح يقرأ if الثانية لأنها مربوطة فيها.

- An **else** is associated with the most recent if that has not been paired with an **else**.

اذا في أكثر من if وحدة كيف نحدد else مربوطة مع أي if ؟ بتكون مع مربوطة مع آخر if قبلها مش مربوطة مع else. مثال:

```
if( y > 50)          //LINE 1
    if( y == 60)     //LINE 2
        cout<<"Pass"; //LINE 3
    else             //LINE 4
        cout<<"Fail"; //LINE 5
else                 //LINE 6
    cout<<"invalid"; //LINE 7
```

نبدأ ب else في line 4 اول if قبلها ما معها else هي if في line 2 ، الثانية else اللي في line 6 اول if قبلها ما معها else هي if في line 1

الكود فوق مكتوب بطريقة مرتبة أسهل للقراءة ، بينما ممكن يكون مكتوب بهذا الشكل في الحالتين لازم نعرف نميز كل `else` مع مين:

```
if( y > 50)           //LINE 1
if( y == 60)          //LINE 2
cout<<"Pass";         //LINE 3
else                  //LINE 4
cout<<"Fail";          //LINE 5
else                  //LINE 6
cout<<"invalid";       //LINE 7
```

EXAMPLE 4-18

Suppose that `balance` and `interestRate` are variables of type `double`. The following statements determine the `interestRate` depending on the value of the `balance`:

```
if (balance > 50000.00)           //Line 1
    interestRate = 0.07;          //Line 2
else                               //Line 3
    if (balance >= 25000.00)       //Line 4
        interestRate = 0.05;      //Line 5
    else                           //Line 6
        if (balance >= 1000.00)    //Line 7
            interestRate = 0.03;   //Line 8
        else                       //Line 9
            interestRate = 0.00;   //Line 10
```

To avoid excessive indentation, the code in Example 4-18 can be rewritten as follows:

```
if (balance > 50000.00)           //Line 1
    interestRate = 0.07;          //Line 2
else if (balance >= 25000.00)      //Line 3
    interestRate = 0.05;          //Line 4
else if (balance >= 1000.00)       //Line 5
    interestRate = 0.03;          //Line 6
else                               //Line 7
    interestRate = 0.00;          //Line 8
```

إذا كانت جملة تنفيذ `else` هي `if statement` بنقدر نستعويض عنها ب `else if` تكون أسهل للقراءة مثل المثال اعلاه.

EXAMPLE 4-19

Assume that `score` is a variable of type `int`. Based on the value of `score`, the following code outputs the grade:

```
if (score >= 90)
    cout << "The grade is A." << endl;
else if (score >= 80)
    cout << "The grade is B." << endl;
else if (score >= 70)
    cout << "The grade is C." << endl;
else if (score >= 60)
    cout << "The grade is D." << endl;
else
    cout << "The grade is F." << endl;
```

Comparing if...else Statements with a Series of if Statements

a.	<pre>if (month == 1) cout << "January" << endl; else if (month == 2) cout << "February" << endl; else if (month == 3) cout << "March" << endl; else if (month == 4) cout << "April" << endl; else if (month == 5) cout << "May" << endl; else if (month == 6) cout << "June" << endl;</pre>	<pre>//Line 1 //Line 2 //Line 3 //Line 4 //Line 5 //Line 6 //Line 7 //Line 8 //Line 9 //Line 10 //Line 11 //Line 12</pre>
b.	<pre>if (month == 1) cout << "January" << endl; if (month == 2) cout << "February" << endl; if (month == 3) cout << "March" << endl; if (month == 4) cout << "April" << endl; if (month == 5) cout << "May" << endl; if (month == 6) cout << "June" << endl;</pre>	



فعليا الكود في a و في b يعطوا نفس ال output ، لكن في a الكود أسرع ليش؟ لانه الكومبايلر طالما تحقق الشرط ما بيقراً كل ال else اللي بعدها لنفرض انه `month == 1` في الكود الاول رح يطبع January وما رح يقرأ كل ال else اللي تحت بينما لو في الكود الثاني رح يطبع January ويشيك على كل if بعدها مما يجعله أبطئ.

Confusion Between `==` and `=`

```
if (x = 5)
```

```
cout << "The value is five." << endl;
```

في هذا المثال في error لكن مش syntax error يعني اذا بتعمل run رح يشتغل عادي طبيعي لكن هذا logical error، لأنه هاي assignment operator مش (==) equal operator اللي كتب الكود قصده يكتبها، الكومبايلر هون رح يعدل قيمة x ويجعلها 5 واللي هو مش الهدف. ملاحظة: هذا الكود بغض النظر عن قيمة x دائما رح يطبع الجملة.

Conditional Operator (?:)

Syntax for using the conditional operator:

expression1 ? expression2 : expression3

(condition ? expression_if_true : expression_if_false;)

مثال:

```
if (mark >= 50)
```

```
cout<<"Pass";
```

```
else
```

```
cout<<"Fail";
```

الكود اعلاه ممكن نكتبه بإستخدام : conditional operator

```
cout << (mark >= 50 ? "Pass" : "Fail") << endl;
```

لاحظوا انه في الحالتين ال statement المستخدمة وحدة وهي cout statement. يعني في هذا المثال تحت ما بنقدر نستعويض عنه ب. conditional operator

```
if ( z % 2 == 0)
```

```
sum+=1;
```

```
else
```

```
cout<<"please try again";
```

switch Structures : alternative to if-else.

switch (integral) expression is evaluated first.

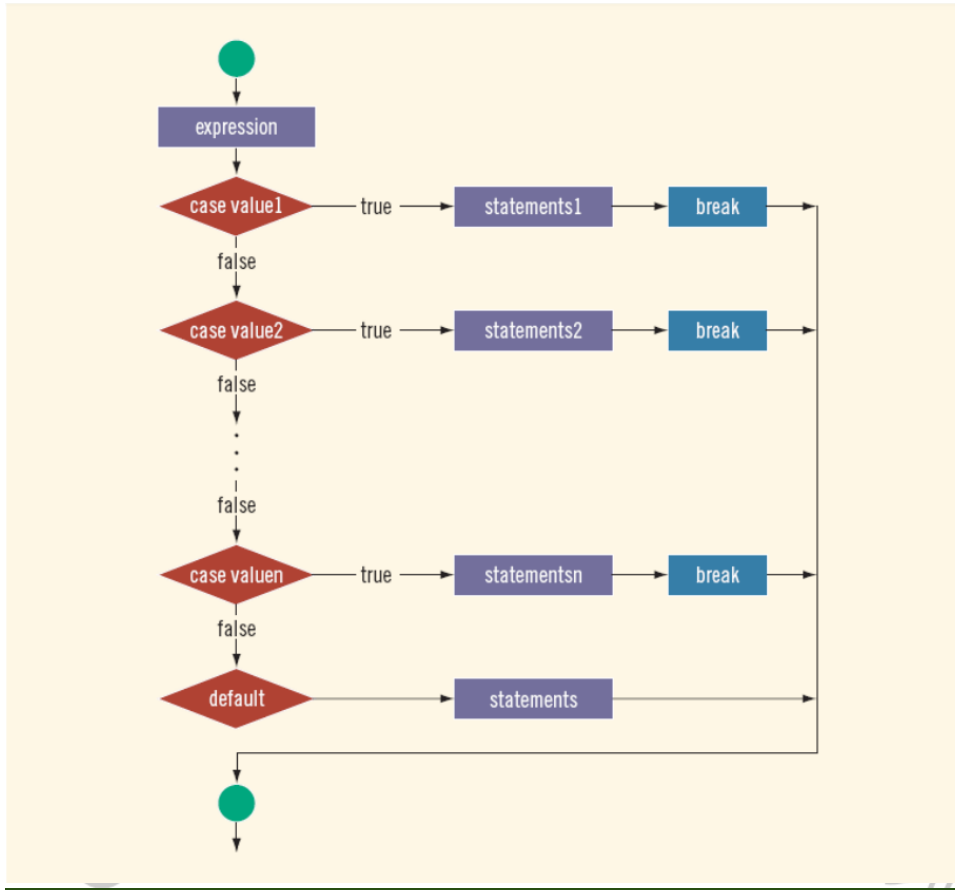
نستخدم switch عندما نريد اتخاذ قرارات متعددة بناءً على قيمة متغير واحد ويجب أن يكون ال variable من نوع (integral (int,bool,char نضع اسم ال variable بين الأقواس.

- Value of the expression determines which corresponding action is taken.

قيمة هذا ال variable هي اللي بتحدد الجمل التي سوف تنفذ.

- Expression is sometimes called the selector

```
switch (expression)
{
    case value1:
        statements1
        break;
    case value2:
        statements2
        break;
    :
    :
    :
    case valuen:
        statementsn
        break;
    default:
        statements
}
```



- One or more statements may follow a `case` label, braces are not needed to turn multiple statements into a single compound statement.
كل case ممكن يكون يتبعها جملة أو أكثر في هذه الحالة لا داعي لل { } braces اذا كان يتبعها اكثر من جملة.
- `break` statement ends switch statement
جملة `break` تنهي switch statement يعني لما يشوفها الكومبايلر بيطلع من switch
- The `break` statement may or may not appear after each statement.
ال `break` ممكن تكون موجودة بعد كل جملة ممكن لأ.
- `switch`, `case`, `break` and `default` are reserved words.

EXAMPLE 4-24

Consider the following statements, where grade is a variable of type `char`:

```
switch (grade)
{
case 'A':
    cout << "The grade is 4.0.";
    break;
case 'B':
    cout << "The grade is 3.0.";
    break;
case 'C':
    cout << "The grade is 2.0.";
    break;
case 'D':
    cout << "The grade is 1.0.";
    break;
case 'F':
    cout << "The grade is 0.0.";
    break;
default:
    cout << "The grade is invalid.";
}
```

ال default statement تنفذ اذا كانت قيمة variable لا تنطبق مع أي من ال cases الموجودة في switch structure.

Problem-Solving Part

examples:

- 1.write a program to check if a number is even or odd.
- 2.write a program to make a calculator.

1.

```
#include <iostream>
using namespace std;
int main()
```

```
{
```

```
    int number;
    cout << "Enter an integer: ";
    cin >> number;
```

```
    if (number % 2 == 0)
        cout << number << " is even." << endl;
    else
        cout << number << " is odd." << endl;
    return 0;
```

```
}
```

2.

```
#include <iostream>
using namespace std;
```

```
int main() {
    double num1, num2;
    char op;

    cout << "Enter first number: ";
    cin >> num1;

    cout << "Enter an operator (+, -, *, /): ";
    cin >> op;

    cout << "Enter second number: ";
    cin >> num2;

    switch (op) {
        case '+':
            cout << "Result: " << num1 + num2 << endl;
            break;
        case '-':
            cout << "Result: " << num1 - num2 << endl;
            break;
        case '*':
            cout << "Result: " << num1 * num2 << endl;
            break;
        case '/':
            if (num2 != 0)
                cout << "Result: " << num1 / num2 << endl;
            else
                cout << "Error: Division by zero is not allowed." << endl;
            break;
        default:
            cout << "Error: Invalid operator." << endl;
            break;
    }

    return 0;
}
```